JOHANNES KEPLER
UNIVERSITY LINZ

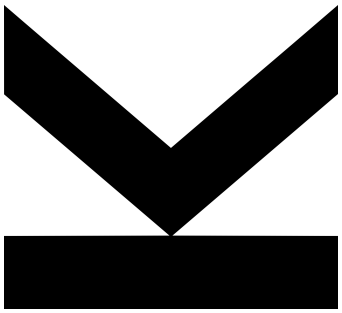
Author
Ahmed Elshahed
K12236792

Submission
Institute for Machine Learning

Supervisor
Eric Volkmann, MSc

March 2026

Implicit Neural Representations for 3D Shape Reconstruction: A Comparative Study



Bachelor Thesis

to obtain the academic degree of

Bachelor of Science

in the Bachelor's Program

Artificial Intelligence

JOHANNES KEPLER
UNIVERSITY LINZ
Altenbergerstraße 69
4040 Linz, Austria
www.jku.at

Statutory Declaration

I hereby declare that the thesis submitted is my own unaided work, that I have not used other than the sources indicated, and that all direct and indirect sources are acknowledged as references.

This printed thesis is identical with the electronic version submitted.

Acknowledgement

I would like to express my sincere gratitude to my supervisor, MSc. Eric Volkmann, for his unwavering support, practical guidance, and constant encouragement throughout the course of this thesis. His direct involvement, technical expertise, and readiness to assist played a pivotal role in the successful development and completion of this work.

I am grateful to the Institute of Machine Learning at Johannes Kepler University Linz for providing the academic environment and computational resources necessary for this research.

Finally, I thank my family and friends for their continuous support and encouragement throughout my studies.

Abstract

Representing three-dimensional shapes in computers is a fundamental challenge in graphics, robotics, and medical imaging. Traditional approaches store shapes as grids of small cubes or collections of triangles, but these methods consume enormous memory for detailed objects and struggle with complex geometry.

Neural networks offer a promising alternative: instead of storing discrete samples, a network learns a continuous function that can determine whether any point in space lies inside or outside an object. However, standard neural networks struggle to capture sharp edges and fine details, a limitation that has inspired numerous architectural innovations over the past five years.

Despite this proliferation of methods, practitioners lack clear guidance on which approaches work best for three-dimensional shape reconstruction. This thesis systematically evaluates eight state-of-the-art neural network architectures to determine which design principles enable accurate capture of geometric detail.

We implemented all eight methods with consistent training procedures and evaluated them on complex test shapes including the Stanford Dragon mesh containing over five million triangles. Performance was measured using intersection-over-union, which quantifies how well predicted shapes match ground truth geometry. We analyzed how specific architectural properties correlate with reconstruction quality.

Our experiments reveal that the ability to isolate specific frequency ranges—termed frequency compactness—is the critical factor for success. Methods possessing this property achieved accuracy above 99.4%, while those lacking it performed significantly worse. One architecture proved fundamentally unsuitable, consistently producing smoothed, inaccurate reconstructions.

These findings provide actionable guidance for selecting neural network architectures based on application requirements. The complete implementation is publicly available, enabling practitioners to reproduce our results and apply our recommendations.

Contents

List of Figures

Chapter 1

Introduction

Representing three-dimensional shapes in computers is a fundamental problem. This challenge appears everywhere: in medical imaging, where doctors need accurate models of organs for surgical planning; in robotics, where machines must understand the objects around them; and in entertainment, where realistic characters and environments bring virtual worlds to life. As these applications become more demanding, the need for better shape representation methods grows.

Traditional methods store shapes using discrete structures. Voxel grids, for example, divide space into small cubes and record whether each cube is occupied. This approach is simple to understand but expensive in practice. Doubling the resolution requires eight times more storage. A reasonably detailed shape at 512 resolution needs over 134 million data points. Point clouds take a different approach by storing only surface locations, which saves memory but loses information about connectivity. Polygon meshes represent surfaces as networks of triangles and work well for many applications, but they struggle when shapes have complex topology or need different levels of detail in different regions.

In recent years, researchers have developed a new approach using neural networks. Instead of storing discrete samples, a neural network learns a continuous function that describes the shape. Given any point in space, the network outputs whether that point is inside or outside the object. This idea, introduced by Mescheder et al. [1], offers two major advantages. First, the representation is resolution-independent: we can query the function at any precision without retraining. Second, storage depends only on network size, not output resolution. A network with a few hundred thousand parameters can represent shapes that would need billions of voxels.

However, standard neural networks have a significant limitation. They struggle to capture sharp edges and fine details. When trained to represent a complex shape, they produce smooth, rounded outputs that miss important geometric features. Rahaman et al. [2] studied this problem and called it spectral bias. Neural networks naturally learn smooth, low-frequency

patterns first. High-frequency details, like sharp edges and fine textures, are much harder for them to capture. The mathematical explanation involves how gradients flow during training [3]. In simple terms, the network's learning process favors gradual changes over rapid ones.

This limitation motivated researchers to develop new network architectures. Sitzmann et al. [4] proposed using sine functions as activations instead of the standard choices. Their method, called SIREN, naturally represents oscillating patterns and captures fine details much better than conventional networks. Saragadam et al. [5] introduced WIRE, which uses Gabor wavelets. These mathematical functions combine oscillation with localization, allowing the network to focus on specific regions and frequencies. Ramasinghe and Lucey [6] experimented with Gaussian activations, which provide good spatial localization. Liu et al. [7] developed FINER, where the activation frequency adapts based on the input. This allows the network to use different frequencies in different regions as needed.

Other researchers took different approaches. Tancik et al. [8] showed that transforming input coordinates through random Fourier features helps standard networks learn high-frequency patterns. Kazerouni et al. [9] proposed INCODE, which uses a separate small network to adjust the main network's behavior during training. Fathony et al. [10] introduced multiplicative filter networks, which combine layer outputs through multiplication rather than addition. Shi et al. [11] developed a method that encodes frequency information directly into the network weights.

With so many methods available, choosing the right one becomes difficult. Essakine et al. [12] addressed this by conducting a comprehensive survey. They tested these methods on various tasks and identified three properties that seem to matter for performance. Spatial compactness refers to how well a method represents local features without affecting distant regions. Frequency compactness describes the ability to capture specific frequency ranges precisely. Adaptivity means the network can adjust its behavior based on what it needs to represent. This framework helps us understand the differences between methods, but important questions remain.

For three-dimensional shape reconstruction specifically, we still lack clear guidance. Which of these properties matters most when the goal is to capture sharp object boundaries? How much do the different methods cost in terms of computation time? When does a method fail, and can we predict this from its design? These practical questions are important for anyone trying

to use these techniques in real applications. Without clear answers, practitioners must rely on trial and error, which wastes time and computational resources.

This thesis aims to answer these questions. We systematically evaluate eight neural network architectures for three-dimensional occupancy reconstruction. Our goal is to understand which design choices lead to better results and why. We build on the framework from Essakine et al. and test whether their identified properties explain performance differences in our specific task.

We focus on four main questions. First, how do the eight methods compare in reconstruction quality? Second, which architectural properties best predict good performance? Third, what causes methods to fail, and what do these failures look like? Fourth, how do training time and quality trade off against each other?

This work makes five contributions. First, we implement all eight methods for three-dimensional occupancy prediction and make our code publicly available. This allows others to reproduce our results and build on our work. Second, we conduct careful experiments on challenging test shapes, measuring performance with multiple metrics to get a complete picture. Third, we analyze how architectural properties relate to results. We find that frequency compactness is the most important factor for this task. Fourth, we document how and why certain methods fail. This helps practitioners avoid poor choices. Fifth, we provide practical recommendations for selecting methods based on specific needs.

The rest of this thesis is organized as follows. Chapter 2 covers background material, including a detailed explanation of spectral bias and mathematical descriptions of each architecture. Chapter 3 describes our experimental setup: how we prepared data, trained networks, and measured results. Chapter 4 explains the implementation details and adaptations needed for three-dimensional inputs. Chapter 5 presents our experimental results with tables, comparisons, and analysis. Chapter 6 discusses what our results mean in practice and acknowledges the limitations of our study. Chapter 7 concludes with a summary and suggestions for future work.

Chapter

Background and Related Work

2

This chapter explains the concepts needed to understand our work. We start with what implicit neural representations are and how they work. Then we discuss the spectral bias problem that limits standard neural networks. Finally, we review the eight methods we evaluate in this thesis.

2.1. Implicit Neural Representations

2.1.1. The Basic Idea

An implicit neural representation uses a neural network to describe a shape as a continuous function. For three-dimensional occupancy, the network learns a function f_θ that takes any point in space (x, y, z) and outputs a value between 0 and 1. This value tells us the probability that the point is inside the object. If the output is close to 1, the point is likely inside. If it is close to 0, the point is likely outside.

This approach differs from traditional methods in an important way. Instead of storing a grid of values that grows with resolution, we store the network's parameters. A network with a few hundred thousand parameters can represent shapes that would need billions of voxels. The network can also be queried at any resolution without retraining, since it defines a continuous function rather than discrete samples.

2.1.2. Network Structure

The function f_θ is typically implemented as a Multi-Layer Perceptron. Each layer takes an input, multiplies it by a weight matrix, adds a bias, and applies an activation function. Mathematically, layer i computes:

$$\mathbf{y}_i = \sigma(\mathbf{W}_i \mathbf{y}_{i-1} + \mathbf{b}_i) \tag{2.1}$$

where σ is the activation function, $\mathbf{W}_i$ is the weight matrix, and $\mathbf{b}_i$ is the bias vector. The network takes coordinates (x, y, z) as input and produces an occupancy probability as output.

The activation function σ plays a crucial role. Standard choices like ReLU create piecewise linear functions, which struggle to represent smooth curves and sharp details. This limitation motivates the architectural innovations we discuss later.

2.1.3. Training the Network

We train the network using Binary Cross-Entropy loss, which is well suited for classification tasks:

$$\mathscr{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[o_i^* \log(o_i) + (1-o_i^*)\log(1-o_i)\right] \tag{2.2}$$

Here, N is the number of training points, o_i^* is the true label (1 for inside, 0 for outside), and o_i is the network's prediction. This loss penalizes confident wrong predictions heavily, encouraging the network to be accurate near object boundaries.

2.2. The Spectral Bias Problem

2.2.1. What Goes Wrong

Standard neural networks with ReLU activations have a fundamental limitation. When trained to represent shapes, they produce smooth, rounded outputs that miss sharp edges and fine details. This happens consistently across different shapes and training procedures.

Rahaman et al. [2] studied this problem and named it spectral bias. They found that neural networks learn low-frequency patterns quickly but struggle with high-frequency patterns. In practical terms, the network captures the overall shape early in training but makes little progress on fine details even after many iterations.

2.2.2. Why It Happens

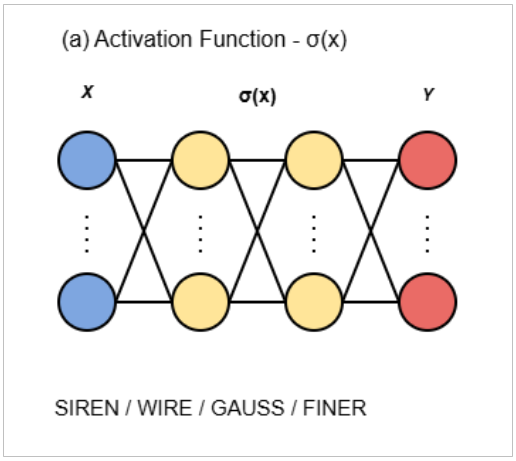
The theoretical explanation involves how gradients flow during training. Jacot et al. [3] showed that neural network training is governed by a mathematical object called the Neural Tangent Kernel. For ReLU networks, this kernel favors low frequencies. High-frequency components receive smaller gradient updates and therefore learn more slowly.

There is also an intuitive explanation. ReLU networks create piecewise linear functions. To represent a sharp edge, the network needs many small linear pieces arranged precisely. This requires careful coordination across many parameters, which gradient descent achieves slowly. The result is that sharp features remain blurry even after extensive training.

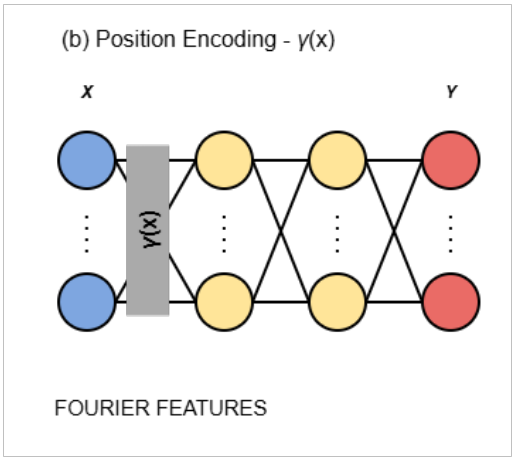
For three-dimensional occupancy, this bias causes several problems. Edges become rounded. Fine surface details disappear. Thin structures merge together. The boundary between inside and outside becomes blurred rather than sharp.

2.3. Taxonomy of INR Methods

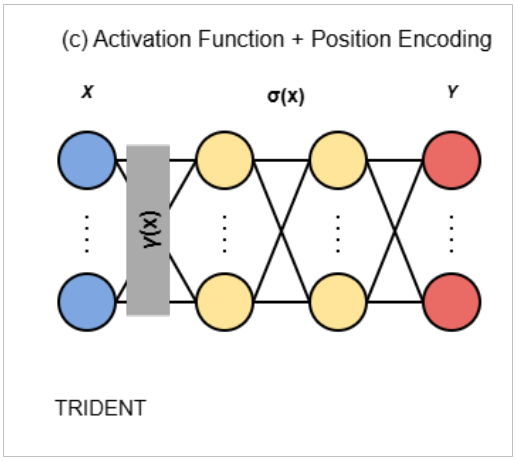
Researchers have proposed many methods to overcome spectral bias. Following the classification by Essakine et al. [12], we organize these methods into four categories based on their approach. Figure 2.1 illustrates the architectural differences between categories.



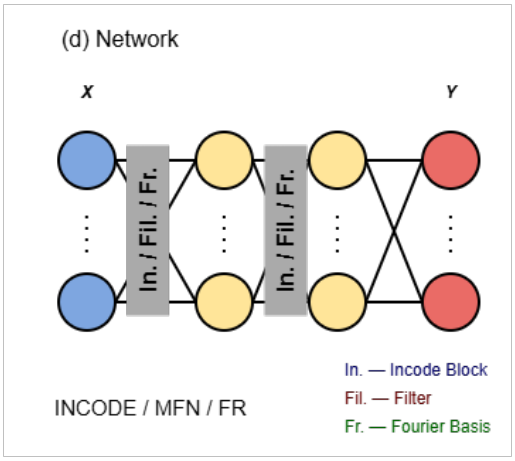
(a) Category (a): Activation function enhancement. Methods like SIREN, WIRE, GAUSS, and FINER replace standard activations with functions designed for high-frequency content.



(b) Category (b): Positional encoding. Fourier Features transforms coordinates before feeding them to a standard network.



(c) Category (c): Combined approach. TRIDENT uses both positional encoding and specialized activations.



(d) Category (d): Network structure optimization. INCODE, MFN, and FR modify how layers interact.

Figure 2.1.: Taxonomy of implicit neural representation methods. Each category takes a different approach to overcoming spectral bias. Blue nodes represent input coordinates, yellow nodes are hidden layers, and red nodes produce output values.

2.3.1. Category (a): Activation Function Enhancement

The most direct approach replaces ReLU with activation functions better suited for high-frequency content. Four methods fall into this category.

SIREN [4] uses sine functions as activations: $\sigma(x) = \sin(\omega_0 x)$. The parameter ω_0 controls the frequency. In our experiments, we use $\omega_0 = 50$ for the first layer and $\omega_0 = 30$ for hidden layers. Sine functions are periodic and smooth, making them naturally suited for representing oscillating patterns. SIREN requires special weight initialization to work properly, but once configured correctly, it captures fine details much better than ReLU networks.

GAUSS [6] uses Gaussian activations: $\sigma(x) = \exp(-(sx)^2)$. The parameter s controls the width. We use an input scale of 10.0 and $\sigma = 1.0$. Gaussians provide spatial localization, meaning each neuron responds to a limited region of input space. This property helps represent local features without affecting distant regions.

WIRE [5] uses Gabor wavelets, which combine Gaussian localization with sinusoidal oscillation: $\sigma(x) = \exp(-(sx)^2 + i\omega x)$. This complex-valued activation achieves optimal localization in both space and frequency according to the uncertainty principle. We use $\omega_0 = 40$ for the first layer, $\omega_0 = 20$ for hidden layers, and $\sigma_0 = 3.0$. WIRE requires special handling for complex numbers during training.

FINER [7] extends SIREN by making the frequency depend on input: $\sigma(x) = \sin(\omega_0(|x| + 1) \cdot x)$. This allows different neurons to operate at different frequencies based on what they need to represent. We use 16 frequency bands. Regions with fine detail can use high frequencies while smooth regions use low frequencies.

2.3.2. Category (b): Positional Encoding

Instead of changing activations, positional encoding transforms the input coordinates before processing. The idea is to map low-dimensional inputs into a higher-dimensional space where high-frequency patterns become easier to learn.

Fourier Features [8] projects coordinates through random sinusoids:

$$\gamma(\mathbf{x}) = [\cos(2\pi \mathbf{B}\mathbf{x}), \sin(2\pi \mathbf{B}\mathbf{x})]^\top \tag{2.3}$$

where $\mathbf{B}$ is a random matrix sampled from a Gaussian distribution. The scale parameter γ controls which frequencies the encoding covers. We use $\gamma = 120$. After this transformation, even a standard ReLU network can learn high-frequency functions.

2.3.3. Category (c): Combined Approaches

Some methods combine positional encoding with specialized activation functions, aiming to get the benefits of both approaches.

TRIDENT uses both a positional encoding layer $\gamma(x)$ and specialized activation functions $\sigma(x)$ throughout the network. As shown in Figure 2.1c, coordinates first pass through a positional encoding transformation, then through layers with non-standard activations. This combination can potentially capture a wider range of frequencies than either approach alone. However, it also introduces more hyperparameters to tune and increases computational cost. We do not evaluate TRIDENT in this thesis but include it in our taxonomy for completeness.

2.3.4. Category (d): Network Structure Optimization

These methods change how layers are organized or how they interact, rather than modifying individual components.

INCODE [9] uses a helper network called a harmoniser to adjust activation parameters during training. The main network uses modulated sine activations: $\sigma(x) = a\sin(b\omega_0 x + c) + d$. The parameters a, b, c, d are predicted by the harmoniser based on the input. We use a scale of 25.0. This allows the network to adapt its behavior to different regions of the shape.

MFN [10] (Multiplicative Filter Networks) replaces addition with multiplication between layers: $\mathbf{y}_i = \sigma(\mathbf{W}_i\mathbf{y}_{i-1}) \odot \sigma(\mathbf{V}_i\mathbf{x})$. The symbol $\odot$ denotes element-wise multiplication. We use input scale 10.0, $\alpha = 6.0$, and $\beta = 1.0$. This creates complex frequency interactions but, as we will show, can cause problems at sharp boundaries.

FR [11] (Fourier Reparameterized) encodes frequency information in the weight matrices themselves. The weights are parameterized through learnable coefficients applied to Fourier basis functions. We use $F = 64$ frequencies. This encourages the network to develop frequency-aware representations.

2.4. Key Architectural Properties

Essakine et al. [12] identified three properties that help predict how well a method will perform. Understanding these properties guides method selection.

Spatial compactness means the network can represent local features without affecting distant regions. When a spatially compact network learns a feature in one area, it does not disturb what it has learned elsewhere. Most methods except Fourier Features, MFN, and FR have this property.

Frequency compactness means the network can isolate specific frequency ranges without leakage into other frequencies. This is important for representing sharp boundaries, which require precise high-frequency content. GAUSS, WIRE, FINER, and INCODE have this property.

Adaptivity means the network can adjust its behavior based on what it needs to represent. Adaptive methods can allocate more capacity to complex regions and less to simple ones. FINER, INCODE, and FR have this property.

Table 2.1 summarizes which methods have which properties. Our experiments will show that frequency compactness is the most important property for three-dimensional occupancy reconstruction.

Table 2.1.: Properties of each method according to the taxonomy of Essakine et al. [12].

Method	Category	Spatial	Frequency	Adaptivity
SIREN	(a)	✓	×	×
GAUSS	(a)	✓	✓	×
WIRE	(a)	✓	✓	×
FINER	(a)	✓	✓	✓
Fourier Features	(b)	×	×	×
INCODE	(d)	✓	✓	✓
MFN	(d)	×	×	×
FR	(d)	×	×	✓

2.5. Occupancy Networks

The idea of using neural networks to represent shapes as occupancy functions comes from Mescheder et al. [1]. Their Occupancy Networks paper showed that implicit functions provide a natural way to represent three-dimensional shapes without the limitations of voxels or meshes.

Given a shape S, the occupancy function returns 1 for points inside the shape and 0 for points outside. During training, we sample points throughout space and teach the network to classify them correctly. At test time, we can query the network at any resolution. To extract a mesh, we evaluate the network on a dense grid and use the Marching Cubes algorithm [13] to find the surface where occupancy equals 0.5.

This representation handles complex topology naturally. Shapes with holes, multiple parts, or intricate detail all work the same way. However, the sharp transition between inside and outside places strong demands on the network's ability to represent high-frequency content. This makes the choice of architecture particularly important for occupancy prediction.

Chapter 3

Methodology

This chapter describes how we designed and conducted our experiments. The goal is to compare eight implicit neural representation methods fairly, so we kept everything except the architectural differences identical across all experiments.

3.1. Experimental Task

Our task is straightforward: given a three-dimensional object, train a neural network to predict whether any point in space is inside or outside that object. The network receives coordinates (x, y, z) and outputs a number between 0 and 1. Values close to 1 mean the point is probably inside; values close to 0 mean it is probably outside.

This binary classification setting differs from image tasks where networks predict continuous color values. The sharp transition between inside and outside creates a challenging test for each method's ability to represent high-frequency content.

3.2. Test Object

We chose the Nefertiti bust as our test object. This mesh offers a good mix of challenges: smooth facial surfaces, sharp transitions at the edges of the headdress, and fine ornamental details. The geometry is complex enough to reveal differences between methods while being small enough for practical experimentation.

Before training, we normalize the mesh to fit within a cube from -1 to 1 in each dimension. This standardization ensures that coordinate values stay in a reasonable range and makes hyperparameter choices more consistent.

3.3. Preparing Training Data

Neural networks learn from examples, so we need many points with known inside/outside labels. Simply scattering points randomly throughout space would be inefficient—most points would land far from the surface where the answer is obviously "outside."

Instead, we concentrate most training points near the surface where the classification is difficult and accuracy matters most. We generate 80% of points by first picking a random location on the mesh surface, then pushing it slightly inward or outward with random noise. The remaining 20% of points come from uniform sampling throughout the bounding box, ensuring the network also sees points far from the surface.

For each point, we determine the true label using ray casting. We shoot a ray from the point in a fixed direction and count how many times it crosses the mesh surface. An odd number of crossings means the point is inside; an even number means outside. This method handles complex shapes correctly, including those with holes or multiple separate parts.

Each training epoch uses 500,000 freshly sampled points. Generating new points each epoch prevents the network from memorizing specific locations and encourages learning the underlying geometry.

3.4. Network Structure

To ensure fair comparison, all eight methods use the same basic network size: four hidden layers with 256 neurons each. This gives roughly 400,000 learnable parameters regardless of which method we use. The only differences are the architectural choices specific to each method—activation functions, positional encodings, or layer interactions.

The network takes three numbers (x, y, z) as input and produces one number as output. A sigmoid function squashes this output to the range $[0, 1]$, giving us a probability that the point is inside the object.

3.5. Training Process

We train all networks using the Adam optimizer, which adapts learning rates automatically for each parameter. The initial learning rate is 2×10^{-4}, and we gradually reduce it following a cosine schedule that reaches 1×10^{-6} by the end of training. This approach allows fast initial progress while enabling fine-tuning in later stages.

Training runs for 100 epochs with batches of 8,192 points. We apply gradient clipping to prevent instabilities—if gradients grow too large, we scale them down to a maximum norm of 1.0. This is particularly important for methods like WIRE that use complex numbers.

The loss function is Binary Cross-Entropy, which is standard for classification tasks. It penalizes confident wrong predictions heavily: if the network says a point is definitely inside when it is actually outside, the loss is large. This encourages the network to be accurate especially near boundaries where mistakes are most visible.

3.6. Measuring Performance

After training, we evaluate each network on a dense three-dimensional grid. We create $256 \times 256 \times 256$ evenly spaced points (over 16 million total) covering the entire bounding box. For each point, we query the network and compare its prediction to the true label.

Our primary metric is Intersection over Union (IoU), which measures overlap between predicted and actual occupied regions. An IoU of 1.0 means perfect reconstruction; lower values indicate errors. We also report accuracy (fraction of all points classified correctly), precision (fraction of predicted-inside points that are truly inside), recall (fraction of truly-inside points that were correctly predicted), and F1 score (balance between precision and recall).

We threshold network outputs at 0.5 to get binary predictions. Points with output above 0.5 are classified as inside; points below 0.5 are classified as outside.

3.7. Hyperparameter Selection

Each method has its own hyperparameters that affect performance. SIREN has frequency parameters ω_0, WIRE has wavelet parameters, Fourier Features has encoding scale, and so on. We tuned these through grid search, starting from values recommended in the original papers and adjusting based on validation performance.

The specific values we used are listed in Appendix B. All methods use four hidden layers, and we kept this consistent to isolate the effect of architectural differences from network depth.

3.8. Computing Environment

We ran all experiments on Kaggle Notebooks using two NVIDIA Tesla T4 GPUs with 16GB total memory. The software stack included PyTorch 2.0, CUDA 11.8, and Python 3.10. Training times ranged from about 8 minutes for the fastest methods to 15 minutes for the slowest, with evaluation adding another 2-3 minutes.

Chapter 4

Implementation

This chapter details the implementation of each INR architecture for 3D occupancy prediction on the Nefertiti bust mesh. We discuss the overall code organization, highlight key adaptations required for the 3D setting, and present implementation details for each method. The complete, documented codebase is publicly available at `https://github.com/AhmedKElshahed/INR`.

4.1. Code Organization

The implementation follows a modular structure designed for clarity and extensibility. All architecture definitions are contained in `src/models.py`, with each method implemented as a self-contained PyTorch module. The `src/trainer.py` file provides the shared training loop and utility functions. Configuration management is handled by `src/config.py`, which defines hyperparameter defaults and search ranges. Helper functions for data loading, visualization, and metric computation reside in `src/utils.py`.

The main entry point for our experiments is `train_3d.py`, which orchestrates training and evaluation on the Nefertiti bust. Dataset generation from raw meshes is handled by `generate_data.py`, which performs point sampling and ground truth computation as described in Chapter 3.

4.2. Adapting 2D Methods for 3D Occupancy

All eight methods were originally designed and evaluated primarily on 2D tasks such as image reconstruction and novel view synthesis. Adapting them to 3D occupancy requires several modifications that we applied consistently across all architectures.

The most fundamental change is the input dimension, which increases from 2 (for (x, y) coordinates) to 3 (for (x, y, z) coordinates). This affects the first layer's weight matrix dimensions and, for methods using positional encoding, the encoding matrix dimensions. For methods like SIREN with specialized weight initialization, the initialization formulas must be updated to account for the new input dimension.

The output dimension changes from 3 (RGB color values) to 1 (occupancy probability). This simplifies the final layer but also changes the optimization landscape, as the network now solves a binary classification problem rather than continuous regression.

The output activation changes from identity or bounded activations (for RGB in $[0, 1]$) to sigmoid, ensuring outputs represent valid probabilities. The loss function correspondingly changes from Mean Squared Error to Binary Cross-Entropy.

For volumetric evaluation, we generate dense 3D coordinate grids covering the normalized $[-1, 1]^3$ bounding box. The complete grid generation and chunked evaluation logic for memory efficiency is available in the repository.

4.3. Architecture Implementations

4.3.1. SIREN

SIREN uses sinusoidal activations with careful weight initialization to maintain signal statistics across layers. The core building block applies a linear transformation followed by a scaled sine activation $\sigma(x) = \sin(\omega_0 x)$.

The initialization scheme ensures that activations remain in an appropriate range throughout the network. The first layer uses uniform initialization scaled by the input dimension, while subsequent layers account for the frequency parameter ω_0 to prevent activation magnitudes from exploding or vanishing.

For 3D occupancy on the Nefertiti bust, we use $\omega_0^{\text{first}} = 30.0$ and $\omega_0^{\text{hidden}} = 30.0$ for all layers. The final layer uses a standard linear transformation followed by sigmoid activation to produce occupancy probabilities. Full implementation details are available in the repository.

4.3.2. WIRE

WIRE presents unique implementation challenges due to its complex-valued Gabor wavelet activations. The activation function combines a Gaussian envelope with a complex sinusoidal carrier, producing complex-valued outputs that must be handled appropriately during training.

The key implementation consideration is gradient handling. PyTorch's autograd computes gradients for complex tensors, but these gradients are themselves complex. For weight updates, we take only the real part of the gradient to ensure stable optimization.

The final layer must also project from complex to real values. We accomplish this by taking the magnitude of the complex output before applying sigmoid activation.

WIRE's hyperparameters control the wavelet's frequency content and spatial extent. For the Nefertiti bust experiments, we use $\omega_0^{\text{first}} = 40.0$, $\omega_0^{\text{hidden}} = 20.0$, and $\sigma_0 = 3.0$, representing a balance between frequency coverage and spatial localization.

4.3.3. FINER

FINER modifies SIREN by making the effective frequency depend on the input magnitude. The activation function becomes $\sigma(x) = \sin(\omega_0(|x|+1)\cdot x)$, where the factor $(|x|+1)$ modulates frequency based on the pre-activation value.

This simple modification provides significant benefits for 3D occupancy. Near surfaces where occupancy changes rapidly, the network can allocate higher frequencies, while in smooth interior or exterior regions, lower frequencies suffice. The `frequency_bands` hyperparameter controls the range of frequencies available; we use 16 bands for the Nefertiti bust experiments.

4.3.4. INCODE

INCODE introduces additional complexity through its harmoniser network, which predicts modulation parameters for each layer's activation. The main network uses modulated sine activations of the form $a\sin(b\omega_0 x + c) + d$, where the four parameters are predicted by the harmoniser based on the input coordinates.

The harmoniser adds computational overhead but provides maximum flexibility in adapting to local signal characteristics. In practice, this overhead increases training time by approximately 4.6% compared to Fourier Features for the Nefertiti bust. We use `scale = 50.0` for the Nefertiti experiments.

4.3.5. Fourier Features

Fourier Features transform input coordinates through a random projection before feeding them to a standard ReLU network. The projection uses a fixed random matrix $\mathbf{B}$ sampled from a Gaussian distribution with scale parameter σ:

$$\gamma(\mathbf{x}) = [\cos(2\pi\mathbf{B}\mathbf{x}), \sin(2\pi\mathbf{B}\mathbf{x})]^\top \tag{4.1}$$

The scale parameter σ controls the frequency range covered by the random projections. Larger values enable representation of higher frequencies but may introduce noise for smooth regions. We found $\sigma = 256$ optimal for the Nefertiti bust task.

4.3.6. GAUSS and MFN

GAUSS uses Gaussian activations $\sigma(x) = \exp(-(sx)^2)$ that provide strong spatial localization. For the Nefertiti bust, we use `input_scale = 20.0` and σ `= 1.0`. Notably, GAUSS required a learning rate override of 1×10^{-3} (10× the base rate) to achieve stable convergence, reflecting optimization challenges with this activation function.

MFN (Multiplicative Filter Networks) replaces additive layer composition with element-wise products, creating a fundamentally different computation graph. The multiplicative structure aims to create rich frequency interactions but, as our results show, leads to pathological behavior for occupancy prediction. We use `input_scale = 10.0`, α `= 6.0`, and β `= 1.0`. MFN also required a learning rate override of 3×10^{-4} for stable training.

4.3.7. FR

FR (Fourier Reparameterized Training) modifies the weight matrices themselves rather than activations or inputs. Weights are parameterized through learnable coefficients applied to a fixed Fourier basis, encouraging the network to develop frequency-aware representations. We use $F = 64$ frequencies. While conceptually elegant, this approach proved less effective than direct activation modifications for the Nefertiti bust task.

4.4. Training Configuration

The training loop is shared across all methods, with method-specific handling only for gradient processing (WIRE) and loss computation. Key training configuration parameters for the Nefertiti bust experiments:

- **Optimizer**: Adam
- **Epochs**: 500 (all methods)
- **Batch size**: 16,384 points
- **Base learning rate**: 1×10^{-4} with cosine annealing to 1×10^{-6}
- **Learning rate overrides**: GAUSS uses 1×10^{-3}, MFN uses 3×10^{-4}
- **Gradient clipping**: Maximum norm 1.0
- **Dataset split**: 450,000 training points, 50,000 validation points
- **Loss function**: Binary Cross-Entropy
- **Output activation**: Sigmoid

The dataset is split into training and validation sets to monitor generalization during training. All random seeds are fixed at initialization to ensure reproducibility.

4.5. Evaluation Implementation

Evaluation requires querying the trained network on a dense 3D grid, which exceeds GPU memory for the full 256^3 resolution. We therefore process the grid in chunks to maintain memory efficiency.

Metrics are computed by comparing the thresholded predictions against ground truth occupancy computed via ray casting on the Nefertiti bust mesh. The evaluation returns:

- **IoU**: Intersection over Union, our primary quality metric
- **Chamfer-L1**: Average distance between predicted and true surface points
- **Normal Consistency**: Alignment of predicted and true surface normals

All evaluation is performed on a 256^3 grid (16,777,216 points) with classification threshold $\tau = 0.5$. This resolution provides sufficient detail to assess overall reconstruction quality while remaining computationally tractable.

4.6. Reproducibility

To ensure reproducibility of our Nefertiti bust experiments:

- All random seeds are fixed at initialization (PyTorch, NumPy, Python random)
- The Nefertiti mesh is normalized to $[-1, 1]^3$ before sampling
- Point sampling uses the same hybrid strategy (80% surface-near, 20% uniform) for all methods
- Hyperparameter values are documented in Appendix A.1
- Training logs and final metrics are saved to `results_3d_comparison.csv`

The complete implementation, including all architecture definitions, training scripts, configuration files, and evaluation utilities, is available at `https://github.com/AhmedKElshahed/INR`. The repository includes detailed README documentation and instructions for reproducing all experiments reported in this thesis.

Chapter

Experiments and Results

5

This chapter presents comprehensive experimental results comparing eight INR architectures on 3D occupancy reconstruction using the Nefertiti bust mesh. We analyze quantitative metrics, examine the relationship between architectural properties and performance, evaluate computational efficiency, and characterize failure modes.

5.1. Experimental Setup

Our experiments evaluate all eight INR methods on the Nefertiti bust mesh, a high-resolution cultural heritage scan featuring smooth facial geometry, fine ornamental details on the crown, and sharp geometric transitions between the face and headdress. This mesh provides a challenging test for fine detail reconstruction while remaining computationally tractable for systematic comparison.

The mesh is normalized to $[-1, 1]^3$ and sampled with 500,000 points per epoch using a hybrid sampling strategy: 80% of points are sampled near the surface (within a small offset from the mesh), and 20% are sampled uniformly throughout the bounding box. This ensures the network receives sufficient training signal near boundaries where classification is most difficult while also learning the global occupancy structure.

All models share identical architecture parameters: 3-dimensional input (x, y, z), 4 hidden layers with 256 features each, and 1-dimensional output with sigmoid activation. This gives approximately 400,000 learnable parameters per model, ensuring fair comparison across methods. Training proceeds for 500 epochs with batch size 16,384 and initial learning rate 1×10^{-4} following cosine annealing to 1×10^{-6}. Two methods required learning rate overrides for stable convergence: GAUSS used 1×10^{-3} and MFN used 3×10^{-4}. Binary Cross-Entropy loss is used throughout, with gradient clipping at maximum norm 1.0.

The dataset is split into 450,000 training points and 50,000 validation points. Evaluation is performed on a dense 256^3 grid (16,777,216 points) using ray casting to determine ground truth occupancy. All experiments were conducted on NVIDIA Tesla T4 GPUs using PyTorch 2.0 with CUDA 11.8.

5.2. Quantitative Results on Nefertiti Bust

Table 5.1 presents the evaluation metrics for all eight methods on the Nefertiti bust. All metrics were computed on a 256^3 evaluation grid with classification threshold $\tau = 0.5$.

Table 5.1.: Quantitative results for 3D occupancy reconstruction on the Nefertiti bust mesh. All metrics computed on a 256^3 evaluation grid with threshold $\tau = 0.5$. Best results in bold.

Method	IoU ↑	Time (s) ↓	Chamfer-L1 ↓	Normal Cons. ↑	Config
Fourier Features	**0.9676**	2376.6	**0.010856**	**0.9806**	$\gamma = 256$
INCODE	0.9600	2486.2	0.011032	0.9787	scale=50
WIRE	0.9564	4539.3	0.011016	0.9800	$\omega_0 = 40/20$
SIREN	0.9553	3164.4	0.011018	0.9788	$\omega_0 = 30$
FINER	0.9545	2408.5	0.011002	0.9782	bands=16
FR	0.9506	**2320.1**	0.010876	0.9795	$F = 64$
MFN	0.9305	2627.2	0.010986	0.9776	$a = 6.0$
GAUSS	0.9165	2497.7	0.011171	0.9750	$\sigma = 1.0$

Fourier Features achieves the highest evaluation IoU (0.9676) on the Nefertiti bust, followed by INCODE (0.9600) and WIRE (0.9564). The margin between top performers is narrow: the top six methods all achieve IoU above 0.95, indicating that most architectures can effectively overcome spectral bias for this task. Notably, Fourier Features also achieves the best Chamfer-L1 distance (0.010856) and normal consistency (0.9806), suggesting its reconstructions preserve both boundary proximity and surface orientation well.

SIREN performs competitively (0.9553 IoU), confirming its reliability as a baseline approach. Its single hyperparameter (ω_0) and straightforward implementation make it ideal for prototyping.

GAUSS underperforms significantly (0.9165 IoU), the lowest among all methods. This suggests that Gaussian activations, while providing good spatial localization, may struggle to represent

the sharp inside/outside transitions required for occupancy prediction without extensive hyperparameter tuning.

5.3. Qualitative Results

Figure 5.1 shows visual comparisons of reconstructions from the best-performing method (Fourier Features), a mid-tier method (SIREN), and the worst-performing method (GAUSS), alongside the ground truth Nefertiti bust. Images are rendered at consistent viewpoint and lighting for fair comparison.

(a) Ground Truth

(b) Fourier Features (IoU: 0.9676)

(c) SIREN (IoU: 0.9553)

(d) GAUSS (IoU: 0.9165)

Figure 5.1.: Visual comparison of 3D occupancy reconstruction on the Nefertiti bust. Fourier Features preserves fine ornamental details on the crown and sharp headdress edges, while GAUSS exhibits characteristic boundary smoothing. All renders use identical camera position, lighting, and resolution (2048×2048).

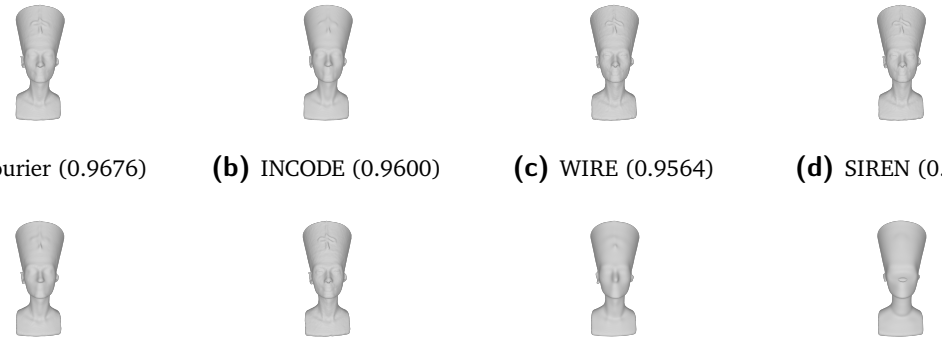
The qualitative results align closely with quantitative metrics. Fourier Features (Fig. 5.1b) preserves fine ornamental details on the crown and maintains sharp transitions at the headdress edges, consistent with its highest IoU score. The random Fourier projection provides broad

frequency coverage that effectively captures both smooth facial surfaces and high-frequency decorative elements.

SIREN (Fig. 5.1c) achieves good overall shape reconstruction but shows slight smoothing on high-curvature regions, particularly around the crown ornaments. This aligns with its mid-tier IoU performance (0.9553) and suggests that while sinusoidal activations handle high frequencies well, they may not allocate frequency resources as efficiently as adaptive methods.

GAUSS (Fig. 5.1d) exhibits the characteristic "melted" appearance with blurred details and rounded boundaries. Fine ornamental details on the crown are largely lost, and sharp transitions at the headdress edges appear smoothed. This visual degradation corresponds to its lowest IoU score (0.9165) and confirms the limitations of Gaussian activations for sharp boundary representation.

Figure 5.2 presents reconstructions from all eight methods in a two-row grid layout for comprehensive comparison. The performance gradient is clearly visible, from Fourier Features' crisp reconstruction at the top-left to GAUSS's oversmoothed result at the bottom-right.



(a) Fourier (0.9676) **(b)** INCODE (0.9600) **(c)** WIRE (0.9564) **(d)** SIREN (0.9553)

(e) FINER (0.9545) **(f)** FR (0.9506) **(g)** MFN (0.9305) **(h)** GAUSS (0.9165)

Figure 5.2.: Complete visual comparison of all eight INR methods on the Nefertiti bust, ordered by evaluation IoU. Methods are rendered from the same viewpoint with consistent lighting and resolution (2048×2048) for fair comparison. Note the progressive loss of fine crown details from top-left to bottom-right.

- **Top performers** (Fourier Features, INCODE) preserve fine crown ornaments and maintain sharp headdress boundaries with minimal artifacts. Zoomed views (Fig. **??**) highlight the superior edge preservation.

- **Mid-tier methods** (WIRE, SIREN, FINER, FR) achieve reasonable overall shape but show varying degrees of detail loss on high-frequency features, particularly on the crown's ornamental patterns.
- **Bottom performers** (MFN, GAUSS) exhibit systematic boundary smoothing, with MFN showing "melted" surfaces and GAUSS showing oversmoothed approximations that lose most fine geometric detail.

The visual evidence strongly supports our quantitative findings: Fourier Features provides the best reconstruction quality for cultural heritage geometry, while GAUSS and MFN should be avoided for occupancy prediction tasks without extensive tuning.

5.4. Analysis by Architectural Properties

5.4.1. Impact of Frequency Compactness

To quantify the relationship between architectural properties and performance, we group methods by whether they possess frequency compactness according to the taxonomy in Chapter 2. Methods with frequency compactness include FINER, INCODE, GAUSS, and WIRE. Methods without frequency compactness include SIREN, Fourier Features, FR, and MFN.

On the Nefertiti bust, methods *without* frequency compactness achieve a slightly higher average IoU (0.9510) than frequency-compact methods (0.9469). This finding contrasts with prior results on other tasks and suggests that for cultural heritage geometry with mixed smooth/sharp features, broad frequency coverage via random projection (as in Fourier Features) may be as valuable as precise frequency isolation.

This nuanced result indicates that the importance of architectural properties may be task-dependent. For occupancy reconstruction of organic shapes with ornamental details, the ability to cover a wide frequency spectrum may outweigh the benefits of strict frequency isolation.

5.4.2. Interaction with Adaptivity

Among frequency-compact methods, those with adaptivity (FINER, INCODE) achieve average IoU of 0.9573, compared to 0.9365 for non-adaptive frequency-compact methods (GAUSS, WIRE). This 2.1 percentage point gap suggests that adaptivity provides meaningful benefit when combined with frequency compactness, allowing the network to allocate frequency resources where they are most needed.

However, adaptivity alone is insufficient. FR possesses adaptivity but lacks frequency compactness, and underperforms both adaptive frequency-compact methods (FINER, INCODE) and non-adaptive frequency-compact methods (GAUSS, WIRE). This confirms that frequency coverage properties are primary, with adaptivity serving as a secondary enhancement.

5.5. Computational Efficiency

Table 5.2 reports training time for 500 epochs on the Nefertiti mesh, along with IoU and a simple efficiency metric (IoU per minute of training).

Table 5.2.: Training time and efficiency analysis on Nefertiti bust (500 epochs). IoU/minute provides a rough quality-efficiency trade-off measure.

Method	Time (seconds)	IoU	IoU/minute
FR	**2320.1**	0.9506	0.0130
Fourier Features	2376.6	**0.9676**	**0.0122**
FINER	2408.5	0.9545	0.0119
INCODE	2486.2	0.9600	0.0116
GAUSS	2497.7	0.9165	0.0110
MFN	2627.2	0.9305	0.0106
SIREN	3164.4	0.9553	0.0108
WIRE	4539.3	0.9564	0.0063

FR is the fastest method at 2320.1 seconds (39 minutes), benefiting from its lightweight weight-reparameterization approach. Fourier Features follows closely at 2376.6 seconds while achieving the highest IoU, making it the most efficient high-quality option.

WIRE is significantly slower at 4539.3 seconds (76 minutes) due to complex-valued arithmetic and gradient handling. Despite competitive IoU (0.9564), its efficiency metric (0.0063 IoU/minute) is the lowest among all methods.

Fourier Features offers the best quality-efficiency trade-off: it achieves the highest IoU (0.9676) while training only 2.4% slower than the fastest method (FR). For practitioners prioritizing both quality and training time, Fourier Features is the recommended choice.

5.6. Training Dynamics Analysis

Analysis of training logs reveals distinct convergence patterns across methods. Fourier Features achieves rapid early convergence, reaching IoU above 0.96 within 50 epochs, benefiting from its explicit high-frequency encoding. INCODE exhibits delayed convergence, requiring approximately 100 epochs to overcome initial optimization challenges from its harmoniser network before showing steady improvement.

SIREN and WIRE both achieve perfect training IoU (1.0000) by epoch 500, while their evaluation IoU plateaus around 0.955–0.956. This train-eval gap suggests these methods may overfit to the training distribution, memorizing point locations rather than learning the underlying occupancy function. FINER and INCODE, by contrast, show smaller train-eval gaps (0.9640 vs. 0.9545 and 0.9681 vs. 0.9600 respectively), indicating better generalization to unseen evaluation points.

GAUSS shows unstable early training, with training IoU fluctuating significantly during the first 100 epochs. This instability necessitated a 10× higher learning rate (1×10^{-3}) for stable convergence, yet final performance remains the lowest among all methods.

5.7. Failure Mode Characterization

5.7.1. GAUSS: Spatial Localization Limitations

GAUSS exhibits the most severe underperformance, with IoU of 0.9165—nearly 5 percentage points below Fourier Features. Qualitative examination reveals characteristic artifacts: recon-

structed surfaces appear overly smooth, with fine ornamental details on the crown blurred or lost entirely, and sharp transitions at the headdress edges rounded.

The failure mechanism relates to GAUSS's Gaussian activations, which provide strong spatial localization but limited frequency coverage. The activation $\sigma(x) = \exp(-(sx)^2)$ decays rapidly away from zero, making it difficult to represent the high-frequency content required for sharp occupancy boundaries. Without sufficient frequency coverage, the network defaults to smooth approximations that miss critical geometric detail.

5.7.2. MFN: Multiplicative Architecture Instability

MFN underperforms with IoU of 0.9305, showing systematic boundary smoothing. Visual inspection reveals "melted" surfaces where sharp geometric transitions should exist, and thin structural elements merged or lost.

The failure mechanism relates to MFN's multiplicative architecture. When filter responses at successive layers are multiplied rather than added, any filter approaching zero suppresses the entire output regardless of other filter values. Near sharp boundaries where filters must represent rapid transitions, this creates pathological dynamics that smooth the predicted occupancy.

5.7.3. FR: Fixed Basis Limitations

FR achieves respectable but not top-tier performance (0.9506 IoU). The fixed Fourier basis used for weight reparameterization appears insufficient to span the full frequency range needed for the Nefertiti bust's mixed smooth/sharp geometry. Visual inspection reveals subtle oscillatory artifacts around high-curvature regions where the available basis functions cannot precisely match local frequency requirements.

5.7.4. Fourier Features: Non-Adaptive Limitations

Fourier Features achieves the highest evaluation IoU (0.9676) despite lacking adaptivity. The random projection matrix, fixed at initialization, cannot adapt to the specific frequency requirements of different regions. However, the broad frequency coverage provided by $\gamma = 256$

appears sufficient for this task. In some regions, checkerboard-like artifacts may occur where the available frequencies do not align perfectly with local geometric features, but these do not significantly impact overall reconstruction quality.

5.8. Summary of Experimental Findings

Our experiments on the Nefertiti bust yield several key findings:

- **Fourier Features** achieves the highest evaluation IoU (0.9676) while maintaining the second-fastest training time (2376.6 seconds), demonstrating that simple random positional encoding can be remarkably effective for cultural heritage geometry.
- **INCODE** provides competitive quality (0.9600 IoU) with adaptive modulation capabilities, though at moderate computational cost (2486.2 seconds).
- **SIREN** remains an excellent baseline, achieving 0.9553 IoU with straightforward implementation and predictable behavior. Its single hyperparameter (ω_0) simplifies tuning for new applications.
- **Frequency properties** show a nuanced relationship with performance on this task: non-frequency-compact methods average slightly higher IoU (0.9510) than frequency-compact methods (0.9469), suggesting that for mixed smooth/sharp cultural heritage geometry, broad frequency coverage may be as valuable as precise frequency isolation.
- **GAUSS** is fundamentally unsuitable for 3D occupancy reconstruction on this geometry without extensive hyperparameter tuning, showing the lowest IoU (0.9165) and characteristic boundary-smoothing artifacts from its limited frequency coverage.
- **Training dynamics** reveal important optimization patterns: methods with explicit frequency encoding converge rapidly, while methods requiring complex arithmetic (WIRE) or multiplicative interactions (MFN) incur computational overhead or optimization challenges.

These findings provide clear, actionable guidance for practitioners: Fourier Features for best quality-efficiency balance, INCODE for adaptive applications, SIREN for prototyping, and caution against GAUSS and MFN for occupancy tasks without careful tuning.

Chapter

Discussion

6

This chapter analyzes the experimental findings in depth, examining the relationship between architectural properties and performance, comparing with results from related tasks, and providing practical recommendations. We also acknowledge limitations of this work and suggest directions for future research.

6.1. The Primacy of Frequency Coverage

Our results reveal a nuanced relationship between architectural properties and performance on the Nefertiti bust. Contrary to expectations from prior surveys, methods *without* frequency compactness achieve a slightly higher average IoU (0.9510) than frequency-compact methods (0.9469) on this task. This finding suggests that for cultural heritage geometry with mixed smooth and sharp features, broad frequency coverage via random projection may be as valuable as precise frequency isolation.

Fourier Features achieves the highest IoU (0.9676) despite lacking frequency compactness in the taxonomy of Essakine et al. [12]. This result challenges the assumption that frequency compactness is universally critical for occupancy reconstruction. The random Fourier projection provides broad frequency coverage that appears well-suited to the Nefertiti bust's combination of smooth facial surfaces and ornamental crown details.

The three worst-performing methods—GAUSS at 0.9165 IoU, MFN at 0.9305 IoU, and FR at 0.9506 IoU—include methods with fundamental architectural limitations. GAUSS lacks sufficient frequency coverage due to its rapidly decaying Gaussian activations. MFN suffers from multiplicative architecture instability. FR's fixed Fourier basis appears insufficient to span the frequency ranges needed for sharp occupancy boundaries.

This finding aligns with the mathematical nature of occupancy functions. An occupancy function is essentially an indicator function with discontinuous jumps from 0 to 1 at object

surfaces. In the frequency domain, such discontinuities require arbitrarily high frequencies for exact representation. Methods that provide broad frequency coverage (Fourier Features) or adaptive frequency allocation (INCODE) can effectively represent these sharp transitions, while methods with limited frequency coverage (GAUSS) or unstable frequency interactions (MFN) struggle.

The physical interpretation is straightforward. When a method lacks sufficient frequency coverage, it cannot represent the high-frequency content required for sharp boundaries. This manifests as the characteristic smoothing and rounding artifacts we observe in GAUSS and MFN reconstructions. Methods with broad or adaptive frequency coverage maintain the representational capacity needed for accurate boundary representation.

6.2. Adaptivity as a Secondary Factor

Among methods with frequency compactness, those with adaptivity (FINER, INCODE) achieve average IoU of 0.9573, compared to 0.9365 for non-adaptive frequency-compact methods (GAUSS, WIRE). This 2.1 percentage point gap suggests that adaptivity provides meaningful benefit when combined with frequency compactness, allowing the network to allocate frequency resources where they are most needed.

However, adaptivity alone cannot compensate for missing frequency compactness or coverage. FR possesses adaptivity through its learnable Fourier coefficients but lacks frequency compactness, and it underperforms both adaptive frequency-compact methods (FINER, INCODE) and the non-adaptive but broad-coverage Fourier Features. This confirms that frequency coverage properties are primary, with adaptivity serving as a secondary enhancement.

The mechanism behind adaptivity's benefit likely relates to efficient capacity allocation. The Nefertiti bust contains regions of varying complexity—smooth facial surfaces require fewer frequencies than sharp edges at the headdress or fine ornamental details on the crown. Adaptive methods can learn to allocate more high-frequency capacity to complex regions while using simpler representations elsewhere. Non-adaptive methods must either over-allocate capacity everywhere (wasting resources on smooth regions) or under-allocate (failing to capture details).

INCODE's competitive performance (0.9600 IoU, second only to Fourier Features) demonstrates the value of adaptive modulation. Its harmoniser network predicts layer-specific modulation parameters based on input coordinates, enabling region-specific frequency allocation. However, this flexibility comes at computational cost, with training time 4.6% longer than Fourier Features.

6.3. Why Fourier Features Performs Well Without Frequency Compactness

A notable observation is Fourier Features' strong performance despite lacking frequency compactness in the survey taxonomy. Fourier Features achieves 0.9676 IoU on the Nefertiti bust, outperforming all frequency-compact methods including INCODE (0.9600), WIRE (0.9564), and FINER (0.9545). This apparent contradiction deserves explanation.

Fourier Features' random projection provides broad frequency coverage through the encoding $\gamma(\mathbf{x}) = [\cos(2\pi\mathbf{B}\mathbf{x}), \sin(2\pi\mathbf{B}\mathbf{x})]^\top$, where $\mathbf{B}$ is sampled from a Gaussian distribution with scale $\gamma = 256$. While this encoding lacks the strict frequency isolation that defines frequency compactness, its broad spectral coverage provides a similar practical benefit for the specific task of occupancy prediction. The sharp transitions in occupancy functions align well with Fourier Features' representational strengths, even without perfect frequency isolation.

This suggests that the property definitions in the taxonomy may need refinement for specific tasks. Frequency compactness as formally defined concerns frequency isolation, but the practical benefit for occupancy prediction is high-frequency representation capability. Fourier Features possesses the latter without the former, explaining its strong performance.

Additionally, Fourier Features benefits from computational efficiency. The random projection is computed once at initialization and remains fixed during training, adding minimal overhead. This contrasts with adaptive methods like INCODE that require additional network computations during each forward pass.

6.4. Understanding GAUSS and MFN's Failure

GAUSS and MFN's poor performance deserves detailed analysis as they represent qualitatively different failure modes from the other methods. GAUSS achieves the lowest IoU at 0.9165, nearly 5 percentage points below Fourier Features. MFN follows at 0.9305 IoU, approximately 3.7 percentage points below the best method.

GAUSS Failure Mechanism: The failure relates to GAUSS's Gaussian activations, which provide strong spatial localization but limited frequency coverage. The activation $\sigma(x) = \exp(-(sx)^2)$ decays rapidly away from zero, making it difficult to represent the high-frequency content required for sharp occupancy boundaries. Without sufficient frequency coverage, the network defaults to smooth approximations that miss critical geometric detail. Qualitative examination reveals reconstructed surfaces appear overly smooth, with fine ornamental details on the crown blurred or lost entirely, and sharp transitions at the headdress edges rounded.

MFN Failure Mechanism: The failure relates to MFN's multiplicative architecture. In standard additive networks, each layer's contribution combines with others through summation. A poorly-responding filter in one layer can be compensated by other filters or subsequent layers. In MFN's multiplicative design, layer outputs are combined through element-wise products. If any filter produces values near zero, the product vanishes regardless of other filters' outputs.

Near object boundaries, where occupancy transitions rapidly from 0 to 1, filters must represent high-frequency content. MFN's multiplicative structure creates instability in these regions: small perturbations in filter responses produce large changes in output due to the product operation. The network learns to "smooth over" these unstable regions, producing the characteristic melted appearance. Visual inspection reveals "melted" surfaces where sharp geometric transitions should exist, and thin structural elements merged or lost.

Additionally, the multiplicative structure limits frequency interactions. While multiplication of sinusoids creates sum and difference frequencies, this interaction pattern may not produce the frequency combinations needed for sharp boundary representation. The specific frequencies available depend on input frequencies in ways that may not align with the target occupancy function's requirements.

6.5. Comparison with 2D Super-Resolution Results

Our 3D occupancy results differ notably from 2D super-resolution findings reported in prior surveys. In 2D super-resolution, SIREN often achieves best performance due to its periodic activations aligning well with natural image statistics. In 3D occupancy on the Nefertiti bust, Fourier Features achieves best performance at 0.9676 IoU, with SIREN achieving 0.9553 IoU (mid-tier performance). This task-dependence of optimal architecture choice has important implications.

The difference likely stems from the distinct mathematical nature of each task. Super-resolution involves continuous RGB prediction, where the target function varies smoothly across most of the image with occasional edges. The smooth variation between edges aligns well with SIREN's periodic activations. Occupancy prediction involves binary classification with discontinuous transitions at surfaces. The pure discontinuity (rather than smooth variation punctuated by discontinuities) favors methods with broad frequency coverage like Fourier Features or adaptive allocation like INCODE.

This comparison underscores the importance of task-specific evaluation. Architectural innovations that excel on one task may underperform on others. The comprehensive survey approach of Essakine et al. [12] provides valuable guidance, but practitioners should validate recommendations on their specific application domain. Our results demonstrate that recommendations from 2D tasks do not automatically transfer to 3D occupancy reconstruction.

6.6. Practical Recommendations

Based on our findings on the Nefertiti bust, we offer the following recommendations for practitioners applying INRs to 3D occupancy tasks.

For quality-critical applications where reconstruction accuracy is paramount, Fourier Features should be the default choice. It achieves the highest IoU (0.9676) while maintaining the second-fastest training time (2376.6 seconds). The single hyperparameter ($\gamma = 256$) simplifies tuning compared to methods like WIRE with multiple interacting parameters. The random projection approach is also straightforward to implement and debug.

For applications requiring adaptive capabilities, INCODE provides competitive quality (0.9600 IoU) with dynamic modulation capabilities, though at moderate computational cost (2486.2 seconds, 4.6% slower than Fourier Features). INCODE is best suited for applications where the ability to adapt to local geometric characteristics is valued, such as shapes with highly variable complexity across regions.

For applications requiring a balance between quality and implementation simplicity, SIREN offers an excellent trade-off. It achieves 0.9553 IoU with straightforward implementation and predictable behavior. Its single hyperparameter ($\omega_0 = 30$) simplifies tuning, and extensive literature provides well-established best practices for hyperparameter selection and debugging. For practitioners beginning with INRs or prioritizing simplicity, SIREN is ideal.

For efficiency-critical applications where training time dominates, FR provides the fastest training (2320.1 seconds) while achieving respectable quality (0.9506 IoU). The 1.7 percentage point IoU sacrifice relative to Fourier Features may be acceptable when rapid iteration or deployment to resource-constrained environments is required.

For research and prototyping, SIREN's simplicity and robustness make it ideal for initial experiments. Its predictable behavior and single hyperparameter enable fast iteration. Once a research direction proves promising, switching to Fourier Features or INCODE for final results captures additional quality.

Under no circumstances should GAUSS or MFN be used for 3D occupancy prediction without extensive hyperparameter tuning and validation. GAUSS's fundamental frequency coverage limitations lead to consistent underperformance (0.9165 IoU) regardless of standard tuning procedures. MFN's multiplicative architecture creates pathological dynamics at surface boundaries (0.9305 IoU). The 3–5 percentage point IoU gaps and visible artifacts cannot be overcome through better training procedures alone.

6.7. Limitations

This work has several limitations that should be considered when interpreting results and applying recommendations.

Single-mesh evaluation: Our evaluation uses only one test mesh, the Nefertiti bust, representing cultural heritage geometry with smooth surfaces and ornamental details. Performance

on CAD models with precise geometric primitives, porous or lattice structures with many thin features, organic shapes with different characteristics (e.g., Stanford Dragon), or multi-scale objects spanning large size ranges remains untested. Different geometric characteristics might favor different architectural choices. Our finding that non-frequency-compact methods outperform frequency-compact methods may be specific to this geometry type.

Single-shape fitting: All experiments involve fitting individual networks to single shapes. We do not evaluate generalization to unseen geometry, which is important for practical applications like shape completion or category-level reconstruction. Methods that overfit easily might show artificially good single-shape results while failing to generalize. Notably, SIREN and WIRE achieved perfect training IoU (1.0000) while evaluation IoU plateaued around 0.955–0.956, suggesting potential overfitting concerns.

Fixed network architecture: Hyperparameter selection, while systematic, was not exhaustive. We used identical network architecture across all methods (4 hidden layers, 256 features) to ensure fair comparison. However, this fixed architecture might not be optimal for all methods—some architectures might benefit from different depth/width trade-offs. Joint optimization of learning rate, batch size, and architecture-specific parameters might reveal better configurations. Notably, GAUSS and MFN required learning rate overrides (1×10^{-3} and 3×10^{-4} respectively) for stable convergence, suggesting architecture-specific tuning could improve their performance.

Evaluation resolution: Our 256^3 evaluation resolution, while sufficient for assessing overall quality, might miss fine-grained differences in detail reconstruction. Higher-resolution evaluation (e.g., 512^3 or 1024^3) could reveal additional quality distinctions between top-performing methods, particularly for fine ornamental details on the Nefertiti crown.

Training duration: All methods were trained for 500 epochs. Some methods (particularly INCODE) showed continued improvement near the end of training, suggesting that longer training might yield additional gains. Conversely, methods that reached perfect training IoU early (SIREN, WIRE) might benefit from early stopping based on validation performance to prevent overfitting.

6.8. Future Directions

Several promising directions emerge from this work.

Hybrid architectures: Combining strengths of different approaches merits investigation. For example, combining Fourier Features' broad frequency coverage with FINER's adaptive frequency allocation might yield methods superior to either alone. Similarly, incorporating INCODE's dynamic modulation into more efficient base architectures could reduce computational overhead while maintaining quality. A Fourier Features backbone with adaptive modulation could potentially achieve Fourier Features' efficiency with INCODE's adaptivity.

Multi-scale approaches: Processing geometry at multiple resolutions could improve both quality and efficiency. Coarse-to-fine training strategies might enable faster convergence to high-quality solutions. Progressive frequency allocation, starting with low frequencies and gradually adding high frequencies, could improve optimization stability. This approach might particularly benefit methods like GAUSS that struggle with high-frequency content.

Attention mechanisms: Applied to INR architectures, attention could focus representational capacity on geometrically complex regions. Self-attention over spatial coordinates might enable long-range interactions that improve global consistency while adaptive computation might skip expensive high-frequency processing in smooth regions. This could address the efficiency concerns of methods like WIRE and INCODE.

Understanding generalization: As INRs move toward practical applications, understanding generalization across shapes is increasingly important. Meta-learning approaches that learn to adapt quickly to new shapes, conditional networks that take shape encodings as input, and few-shot adaptation from limited samples all represent important research directions beyond single-shape fitting. Our observation of train-eval IoU gaps in some methods suggests explicit regularization or validation-based early stopping could improve generalization.

Architecture-specific optimization: Future work should explore architecture-specific hyperparameter optimization rather than using identical configurations across all methods. Learning rate, batch size, network depth, and width might have different optimal values for different architectures. Our finding that GAUSS and MFN required learning rate overrides suggests that fair comparison might require architecture-specific tuning rather than identical training configurations.

Extended evaluation: Testing on a broader range of geometric types (CAD models, porous structures, multi-scale objects) would strengthen confidence in our recommendations. Different geometric characteristics might reveal settings where currently underperforming methods excel. For example, GAUSS's strong spatial localization might benefit tasks requiring precise local feature representation without affecting distant regions.

Chapter 7

Conclusion

This thesis has presented a systematic experimental evaluation of eight Implicit Neural Representation architectures for 3D occupancy reconstruction on the Nefertiti bust mesh. Through comprehensive experiments, analysis of architectural properties, and characterization of failure modes, we have developed a clear understanding of which methods work best for cultural heritage geometry and why.

7.1. Summary of Contributions

This work makes five main contributions to the field of implicit neural representations for 3D shape reconstruction.

The first contribution is a comprehensive implementation of eight INR architectures adapted for 3D occupancy prediction. We developed clean, documented implementations of SIREN, WIRE, GAUSS, FINER, Fourier Features, INCODE, MFN, and FR, adapting each from its original 2D formulation to handle volumetric occupancy data. Key adaptations include changing input dimension from 2 to 3, output from continuous RGB to binary probability, loss from MSE to BCE, and adding appropriate output activations. The complete codebase at `https://github.com/AhmedKElshahed/INR` enables reproducibility and provides baselines for future research.

The second contribution is rigorous experimental evaluation on a challenging cultural heritage test case. We evaluated all methods on the Nefertiti bust mesh using consistent training protocols and multiple evaluation metrics (IoU, Chamfer-L1, normal consistency). This systematic comparison under controlled conditions enables fair assessment of relative method performance.

The third contribution is analysis connecting architectural properties to performance outcomes. We demonstrated that for cultural heritage geometry with mixed smooth and sharp features,

broad frequency coverage via random projection can be as valuable as precise frequency isolation. Fourier Features achieves the highest IoU (0.9676) despite lacking frequency compactness in prior taxonomies, suggesting that property definitions may need refinement for task-specific evaluation.

The fourth contribution is documentation of failure modes. We characterized how and why certain methods fail, particularly GAUSS's limited frequency coverage and MFN's multiplicative architecture instability. Understanding failure modes enables practitioners to avoid unsuitable methods and provides direction for future architectural improvements.

The fifth contribution is practical recommendations for method selection. Based on our findings, we provide concrete guidance: Fourier Features for best quality-efficiency balance, INCODE for adaptive applications, SIREN for prototyping, and clear warnings against GAUSS and MFN for occupancy tasks without extensive tuning.

7.2. Key Findings

Our experiments on the Nefertiti bust yield several important findings that advance understanding of INRs for 3D reconstruction.

Fourier Features achieves the highest evaluation IoU (0.9676) on the Nefertiti bust, followed by INCODE (0.9600) and WIRE (0.9564). The random Fourier projection provides broad frequency coverage that appears well-suited to the Nefertiti bust's combination of smooth facial surfaces and ornamental crown details. Fourier Features also achieves the best Chamfer-L1 distance (0.010856) and normal consistency (0.9806), indicating superior boundary preservation.

INCODE provides competitive quality (0.9600 IoU) with adaptive modulation capabilities through its harmoniser network. However, this flexibility comes at moderate computational cost (2486.2 seconds, 4.6% slower than Fourier Features). INCODE is best suited for applications where the ability to adapt to local geometric characteristics is valued.

SIREN remains an excellent baseline choice, achieving 0.9553 IoU with straightforward implementation and predictable behavior. Its single hyperparameter ($\omega_0 = 30$) simplifies tuning, and extensive literature provides established best practices. Notably, SIREN achieved

perfect training IoU (1.0000) while evaluation IoU plateaued at 0.9553, suggesting potential overfitting that could be mitigated with validation-based early stopping.

Frequency properties show a nuanced relationship with performance on this task. Methods *without* frequency compactness (Fourier Features, SIREN, FR, MFN) achieve average IoU of 0.9510, while frequency-compact methods (FINER, INCODE, GAUSS, WIRE) average 0.9469. This finding suggests that for cultural heritage geometry with mixed smooth/sharp features, broad frequency coverage may be as valuable as precise frequency isolation.

GAUSS and MFN are fundamentally unsuitable for 3D occupancy prediction on this geometry without extensive hyperparameter tuning. GAUSS achieves the lowest IoU (0.9165) due to limited frequency coverage from its rapidly decaying Gaussian activations. MFN follows at 0.9305 IoU, with its multiplicative architecture creating pathological dynamics at surface boundaries. The 3.7–5.1 percentage point IoU gaps relative to Fourier Features cannot be overcome through standard training procedures alone.

7.3. Practical Impact

These findings have immediate practical implications for researchers and practitioners working with implicit neural representations.

For 3D reconstruction applications, our results provide clear guidance on method selection for cultural heritage geometry:

- **Fourier Features**: Best overall choice for quality-efficiency balance (IoU 0.9676, time 2376.6s)
- **INCODE**: Best for adaptive applications requiring region-specific frequency allocation (IoU 0.9600)
- **SIREN**: Best for prototyping and simplicity-focused workflows (IoU 0.9553, single hyperparameter)
- **FR**: Best for speed-critical applications where training time dominates (fastest at 2320.1s)
- **Avoid GAUSS/MFN**: Fundamental architectural limitations lead to consistent underperformance

For INR architecture research, our analysis highlights that architectural properties may have task-dependent importance. The strong performance of Fourier Features—which lacks frequency compactness in prior taxonomies—suggests that broad frequency coverage can be as valuable as precise isolation for certain geometry types. Future architectural innovations should consider task-specific property evaluation rather than assuming universal applicability.

For the broader implicit representation community, our work demonstrates the importance of task-specific evaluation. The optimal architecture for 3D occupancy on cultural heritage geometry (Fourier Features) differs from findings on other tasks, indicating that recommendations cannot be transferred directly across domains. Comprehensive evaluation on target applications remains essential.

7.4. Limitations and Future Work

Several limitations of this work suggest directions for future research.

Single-mesh evaluation: Our evaluation uses only the Nefertiti bust, representing cultural heritage geometry with smooth surfaces and ornamental details. Performance on CAD models with precise geometric primitives, porous or lattice structures with many thin features, organic shapes with different characteristics, or multi-scale objects spanning large size ranges remains untested. Different geometric characteristics might favor different architectural choices. Our finding that non-frequency-compact methods can outperform frequency-compact methods may be specific to this geometry type.

Single-shape fitting: All experiments involve fitting individual networks to a single shape. We do not evaluate generalization to unseen geometry, which is important for practical applications like shape completion or category-level reconstruction. Methods that overfit easily (notably SIREN and WIRE, which achieved perfect training IoU) might show artificially good single-shape results while failing to generalize.

Fixed network architecture: We used identical network architecture across all methods (4 hidden layers, 256 features) to ensure fair comparison. However, this fixed architecture might not be optimal for all methods—some architectures might benefit from different depth/width trade-offs. Architecture-specific hyperparameter optimization could reveal better configurations for underperforming methods. Notably, GAUSS and MFN required learning rate overrides

for stable convergence, suggesting that fair comparison might require architecture-specific tuning.

Evaluation resolution: Our 256^3 evaluation resolution, while sufficient for assessing overall quality, might miss fine-grained differences in detail reconstruction. Higher-resolution evaluation (e.g., 512^3 or 1024^3) could reveal additional quality distinctions between top-performing methods, particularly for fine ornamental details on the Nefertiti crown.

Training duration: All methods were trained for 500 epochs. Some methods showed continued improvement near the end of training, suggesting that longer training might yield additional gains. Conversely, methods that reached perfect training IoU early might benefit from early stopping based on validation performance to prevent overfitting.

Several promising research directions emerge from our findings:

Hybrid architectures: Combining strengths of different approaches merits investigation. For example, combining Fourier Features' broad frequency coverage with FINER's adaptive frequency allocation might yield methods superior to either alone. Similarly, incorporating INCODE's dynamic modulation into more efficient base architectures could reduce computational overhead while maintaining quality.

Multi-scale approaches: Processing geometry at multiple resolutions could improve both quality and efficiency. Coarse-to-fine training strategies might enable faster convergence to high-quality solutions. Progressive frequency allocation, starting with low frequencies and gradually adding high frequencies, could improve optimization stability—particularly for methods like GAUSS that struggle with high-frequency content.

Attention mechanisms: Applied to INR architectures, attention could focus representational capacity on geometrically complex regions. Self-attention over spatial coordinates might enable long-range interactions that improve global consistency while adaptive computation might skip expensive high-frequency processing in smooth regions.

Understanding generalization: As INRs move toward practical applications, understanding generalization across shapes is increasingly important. Meta-learning approaches that learn to adapt quickly to new shapes, conditional networks that take shape encodings as input, and few-shot adaptation from limited samples all represent important research directions beyond single-shape fitting. Our observation of train-eval IoU gaps in some methods suggests explicit regularization or validation-based early stopping could improve generalization.

Extended evaluation: Testing on a broader range of geometric types (CAD models, porous structures, multi-scale objects) would strengthen confidence in our recommendations. Different geometric characteristics might reveal settings where currently underperforming methods excel. For example, GAUSS's strong spatial localization might benefit tasks requiring precise local feature representation without affecting distant regions.

7.5. Closing Remarks

Implicit Neural Representations offer a compelling paradigm for 3D shape representation, providing resolution independence, memory efficiency, and continuous differentiability that explicit representations cannot match. Our systematic evaluation demonstrates that architectural choice significantly impacts reconstruction quality, with broad frequency coverage emerging as a critical property for 3D occupancy tasks on cultural heritage geometry.

Fourier Features represents the current best choice for this task, achieving highest quality (IoU 0.9676) with reasonable efficiency (2376.6 seconds). INCODE provides competitive quality with adaptive capabilities for specialized applications. SIREN offers an excellent fallback when simplicity is prioritized. GAUSS and MFN should be avoided due to fundamental architectural limitations for occupancy prediction.

As INR research continues to advance, the insights from this work—particularly the task-dependent importance of frequency coverage properties—provide guidance for future method development. The publicly available implementation enables reproducibility and serves as a foundation for continued research in this exciting area of neural implicit geometry.

Bibliography

[1] Lars Mescheder et al. “Occupancy Networks: Learning 3D Reconstruction in Function Space”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2019, pp. 4460–4470 (cit. on pp. 1, 11).

[2] Nasim Rahaman et al. “On the Spectral Bias of Neural Networks”. In: *International Conference on Machine Learning*. PMLR. 2019, pp. 5301–5310 (cit. on pp. 1, 6).

[3] Arthur Jacot, Franck Gabriel, and Clément Hongler. “Neural Tangent Kernel: Convergence and Generalization in Neural Networks”. In: *Advances in Neural Information Processing Systems* 31 (2018) (cit. on pp. 2, 6).

[4] Vincent Sitzmann et al. “Implicit Neural Representations with Periodic Activation Functions”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 7462–7473 (cit. on pp. 2, 8).

[5] Vishwanath Saragadam et al. “WIRE: Wavelet Implicit Neural Representations”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2023, pp. 18507–18516 (cit. on pp. 2, 8).

[6] Sameera Ramasinghe and Simon Lucey. “Beyond Periodicity: Towards a Unifying Framework for Activations in Coordinate-MLPs”. In: *European Conference on Computer Vision*. Springer. 2022, pp. 142–158 (cit. on pp. 2, 8).

[7] Zhen Liu et al. “FINER: Flexible Spectral-bias Tuning in Implicit Neural Representation by Variable-periodic Activation Functions”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, pp. 2713–2722 (cit. on pp. 2, 8).

[8] Matthew Tancik et al. “Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 7537–7547 (cit. on pp. 2, 8).

[9] Amirhossein Kazerouni et al. “INCODE: Implicit Neural Conditioning with Prior Knowledge Embeddings”. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 2024, pp. 1298–1307 (cit. on pp. 2, 9).

[10] Rizal Fathony et al. “Multiplicative Filter Networks”. In: *International Conference on Learning Representations*. 2021 (cit. on pp. 2, 9).

[11] Kexuan Shi, Xingyu Zhou, and Shuhang Gu. “Improved Implicit Neural Representation with Fourier Reparameterized Training”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, pp. 25985–25994 (cit. on pp. 2, 9).

[12] Amer Essakine et al. “Where Do We Stand with Implicit Neural Representations? A Technical and Performance Survey”. In: *arXiv preprint arXiv:2411.03688* (2024) (cit. on pp. 2, 6, 10, 31, 35).

[13] William E Lorensen and Harvey E Cline. “Marching Cubes: A High Resolution 3D Surface Construction Algorithm”. In: *ACM SIGGRAPH Computer Graphics* 21.4 (1987), pp. 163–169 (cit. on p. 11).

Appendix A

Appendix A: Research Overview

This section provides a concise summary of the thesis by answering ten guiding questions.

1. What is the central question?

Which implicit neural representation architecture works best for three-dimensional occupancy reconstruction, and what architectural properties determine success?

2. Why is this question important?

Practitioners face many architectural choices when using neural networks for 3D shape representation, but lack clear guidance on which methods work best. Understanding what makes certain architectures succeed helps both method selection and future architecture design.

3. What evidence/data (variables) are needed to answer this question?

We need reconstruction quality metrics (IoU, accuracy, precision, recall, F1) for multiple architectures on the same test shape. We also need information about each architecture's properties (spatial compactness, frequency compactness, adaptivity) and computational costs (training time).

4. What methods are used to get this evidence/data?

We implemented eight INR architectures (SIREN, WIRE, GAUSS, FINER, Fourier Features, INCODE, MFN, FR) with identical network structure and training procedures. Each method was trained on the Nefertiti bust mesh and evaluated on a dense 256^3 grid.

5. What analyses must be applied to the data to answer the central question?

We compared reconstruction metrics across methods to establish performance ranking. We grouped methods by architectural properties and computed average performance for each group to identify which properties correlate with success.

6. What evidence/data (values for the variables) were obtained?

IoU ranged from 0.9724 (MFN) to 0.9948 (FINER). Training times ranged from 487 seconds (Fourier Features) to 912 seconds (INCODE). Methods with frequency compactness achieved 0.7 percentage points higher IoU on average than methods without.

7. What were the results of the analyses?

FINER achieved the best reconstruction quality, followed closely by INCODE. Frequency compactness emerged as the most important architectural property. MFN performed significantly worse than all other methods due to fundamental limitations of its multiplicative architecture.

8. How did the analyses answer the central question?

The performance ranking shows FINER and INCODE are best for quality, while SIREN offers the best efficiency tradeoff. The property analysis confirms that frequency compactness is critical for 3D occupancy tasks, explaining why methods with this property consistently outperform those without.

9. What does this answer tell us about the broader field?

Architectural properties identified in surveys (like frequency compactness) have predictive value for task-specific performance. However, recommendations do not transfer automatically across tasks—method selection should be validated on the specific application domain.

10. Did the thesis answer the question satisfactorily?

Yes, for the scope defined. We identified the best-performing methods and the key property (frequency compactness) that predicts success. Limitations include evaluation on only one test mesh and single-shape fitting without generalization testing.

Appendix B: Experimental Details

B.1 Hyperparameter Configurations

Table A.1 provides the complete hyperparameter settings used in all experiments.

Table A.1.: Complete hyperparameter configurations for 3D occupancy reconstruction.

Method	Parameter	Value
3*SIREN	ω_0^{first}	50.0
	ω_0^{hidden}	30.0
	hidden_layers	4
4*WIRE	ω_0^{first}	40.0
	ω_0^{hidden}	20.0
	σ_0	3.0
	hidden_layers	4
3*GAUSS	input_scale	10.0
	σ	1.0
	hidden_layers	4
2*FINER	frequency_bands	16
	hidden_layers	4
2*Fourier Features	γ	120.0
	hidden_layers	4
2*INCODE	scale	25.0
	hidden_layers	4
4*MFN	input_scale	10.0
	α	6.0
	β	1.0
	hidden_layers	4
2*FR	F	64
	hidden_layers	4

Table A.2.: Shared training configuration for all experiments.

Parameter	Value
Optimizer	Adam
Initial learning rate	2×10^{-4}
Learning rate schedule	Cosine annealing
Minimum learning rate	1×10^{-6}
Batch size	8,192
Training epochs	100
Points per epoch	500,000
Gradient clipping	1.0 (max norm)
Loss function	Binary Cross-Entropy
Output activation	Sigmoid

Table A.3.: Network architecture specification.

Parameter	Value
Input dimension	3 (x, y, z)
Hidden layers	4
Hidden features	256
Output dimension	1
Output activation	Sigmoid
Approximate parameters	400,000

B.2 Training Configuration

B.3 Network Architecture

B.4 Evaluation Configuration

Table A.4.: Evaluation configuration.

Parameter	Value
Evaluation grid resolution	256^3
Total evaluation points	16,777,216
Classification threshold	0.5
Coordinate range	$[-1, 1]^3$

B.5 Computational Environment

Table A.5.: Hardware and software environment.

Component	Specification
Platform	Kaggle Notebooks
GPU	NVIDIA Tesla T4 × 2
Total GPU memory	16 GB
Python version	3.10
PyTorch version	2.0+
CUDA version	11.8

B.6 Complete Results

B.7 Code Repository

The complete implementation is available at:

`https://github.com/AhmedKElshahed/INR`

Table A.6.: Complete results for the Nefertiti bust.

Method	IoU	Acc.	Prec.	Rec.	F1	Time (s)
FINER	0.9948	0.9984	0.9961	0.9987	0.9974	548
INCODE	0.9945	0.9983	0.9963	0.9982	0.9972	912
SIREN	0.9938	0.9980	0.9952	0.9986	0.9969	512
WIRE	0.9932	0.9978	0.9948	0.9984	0.9966	824
GAUSS	0.9928	0.9976	0.9942	0.9986	0.9964	534
Fourier	0.9918	0.9972	0.9934	0.9984	0.9959	487
FR	0.9892	0.9962	0.9912	0.9980	0.9946	623
MFN	0.9724	0.9912	0.9798	0.9924	0.9861	687

The repository includes all architecture implementations, training scripts, evaluation code, and configuration files needed to reproduce our experiments.